

FLOATING POINT NUMBERS

SYED HASAN SAEED

FLOATING POINT NUMBERS

REFERENCE BOOKS

- ❖ Digital Systems, Principle & Applications, Ronald J. Tocci, Prentice-Hall
- ❖ Digital Design, M. Morris Mano, Michael D. Ciletti, Pearson Education, Inc.
- ❖ Digital Circuits and Design, S. Salivahanan, S. Arivazhagan, Oxford University Press.
- ❖ Digital Electronics, G. K. Kharate, Oxford University Press.
- ❖ Digital Electronics, Bignell James, Logic and Systems, Cengage Learning
- ❖ Digital Logic and Computer Design, M. Morris Mano, Pearson Education, Inc.

Declaration : The content of the presentation has been created for academic use / non-commercial use with fair dealing.

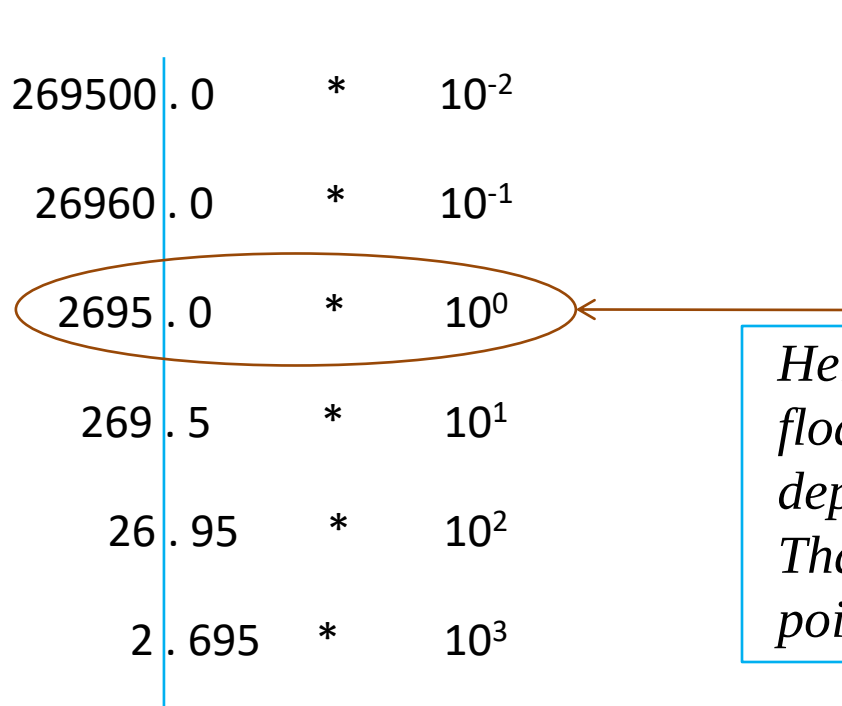
FLOATING POINT NUMBERS

EXPONENTIAL NOTATION:

In the decimal system, very large and very small numbers are expressed in scientific notation.

The following are equivalent representation of 2,695

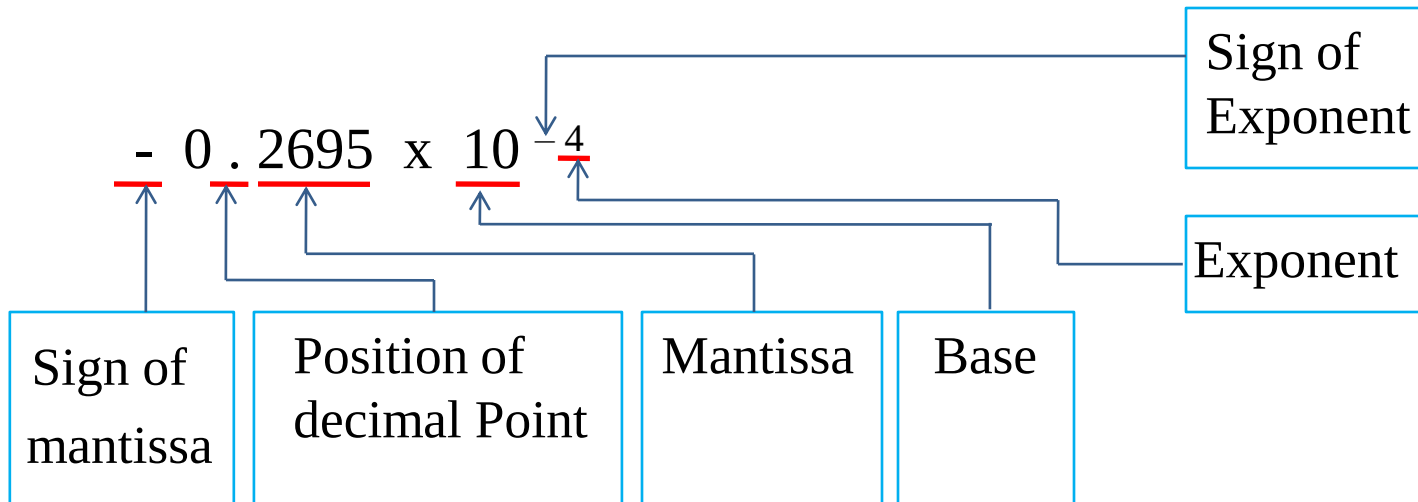
269500	.	0	*	10^{-2}
26960	.	0	*	10^{-1}
2695	.	0	*	10^0
269	.	5	*	10^1
26	.	95	*	10^2
2	.	695	*	10^3



Here decimal is not fixed, it floats to the left or right depending upon our requirement. That is why it is called floating point number system.

FLOATING POINT NUMBERS

- Binary numbers can also be expressed by floating point representation.
- The floating point representation of a number consists of following parts



FLOATING POINT NUMBERS

COMMON STANDARD FOR REPRESENTING FLOATING POINT NUMBERS:

1. Single Precision consist of

- (i) Sign bit (1 bit)
- (ii) Exponent (8 bits)
- (iii) Mantissa (23 bits)

} 32 Bits

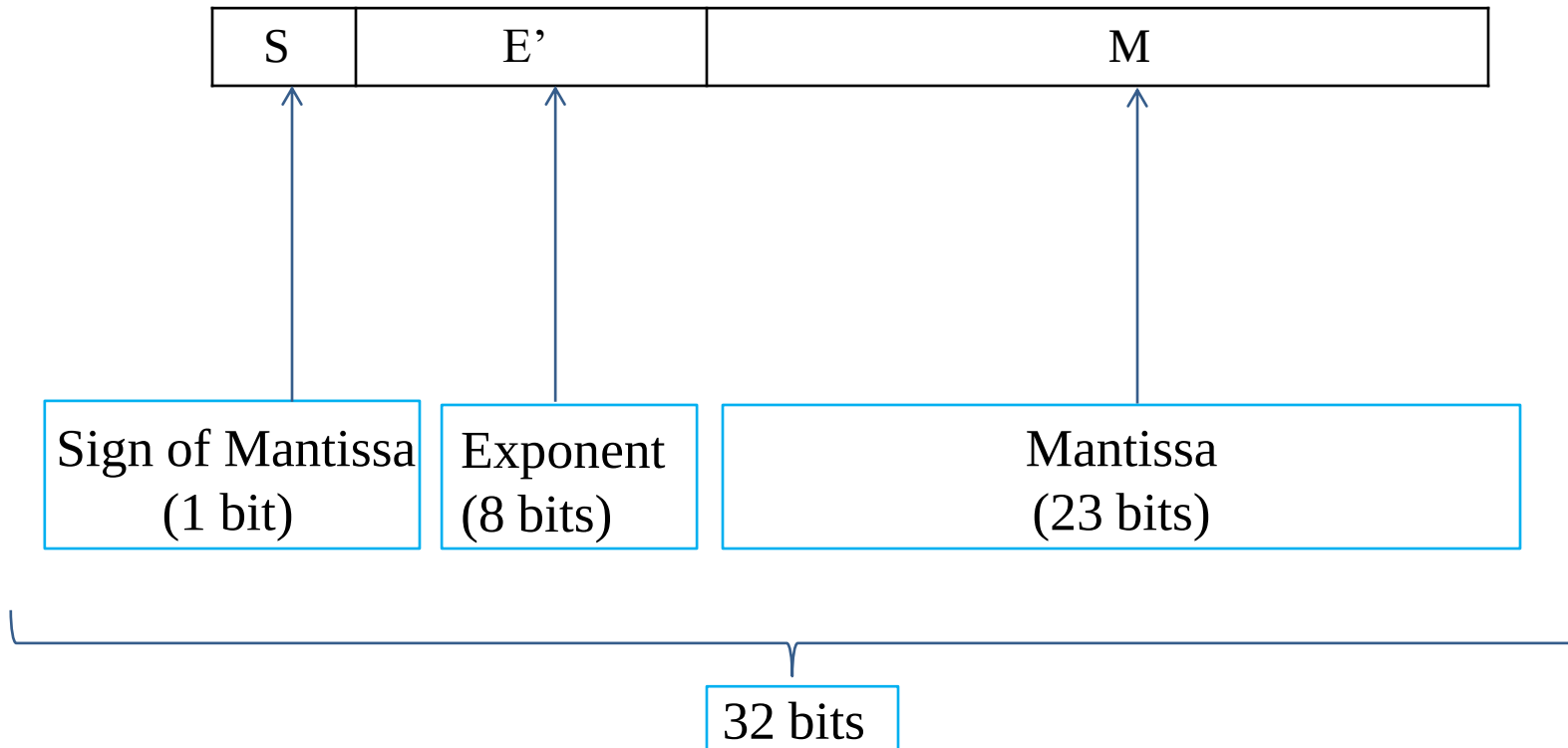
2. Double Precision consists of

- (i) Sign bit (1 bit)
- (ii) Exponent (11 bits)
- (iii) Mantissa (52 bits)

} 64 Bits

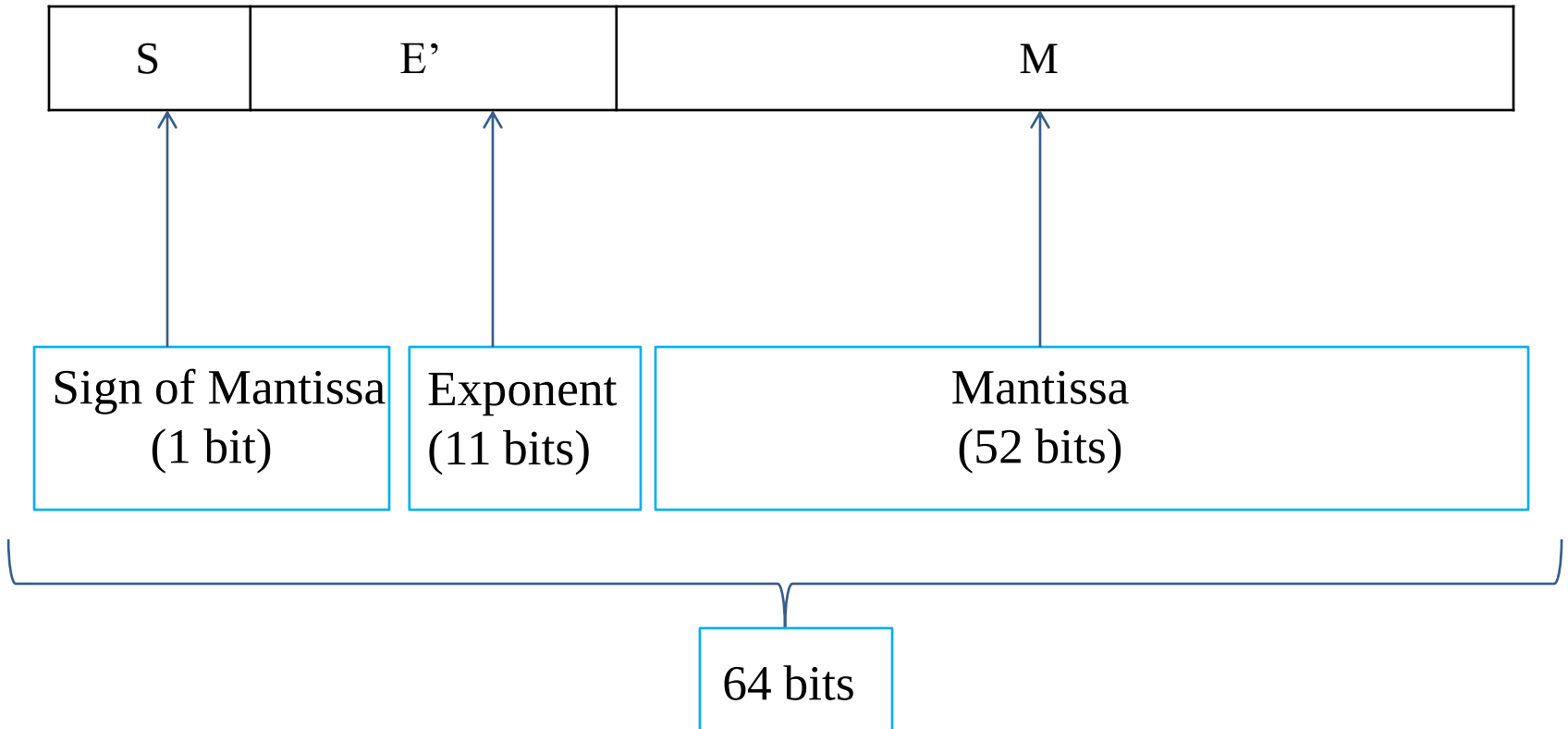
FLOATING POINT NUMBERS

SINGLE PRECISION FORMAT:



FLOATING POINT NUMBERS

DOUBLE PRECISION FORMAT:



FLOATING POINT NUMBERS

NORMALIZATION:

Normalization is the process of moving the binary point so that the first digit after the point is a significant digit. This maximizes precision in a given number of bits.

M : Normalize mantissa, remove the leading (left most) bit, since it always 1 (and the decimal point, if the case) then adjust its length to 23 or 52 bits, by removing the excess bits from right (if any of the excess bits is set on 2, we are losing precision...).

E: 8/ 11 bits are used for Exponent. E represents signed binary integer, so exponent value can range from -128 to +127.

FLOATING POINT NUMBERS

- The sign of mantissa is 0 for positive or 1 for negative.
- In the mantissa, the decimal point is assumed to follow the first '1'. Since the first digit is always a '1', a hidden bit is used to representing the bit. The fraction is the 23 bits following the first '1'. The fraction really represents a 24 bit mantissa.
- The exponent field has a bias of 127, meaning that 127 is added to the exponent before it's stored.

Reference : Department of Computer and Information Science, School of Science, IUPUI

FLOATING POINT NUMBERS

Floating point numbers are standardized by IEEE. According to it we have to represent floating point number in the form

$$\pm 1 . M \times 2^{E' - 127} \quad (\text{Single Precision} - 32 \text{ bit})$$

$$\pm 1 . M \times 2^{E' - 1023} \quad (\text{Double Precision} - 64 \text{ bit})$$

M – Mantissa , E - Exponent Number

$$E = E' - 127 \quad \text{or} \quad E' = E + 127 \quad (\text{Single Precision})$$

$$E = E' - 1023 \quad \text{or} \quad E' = E + 1023 \quad (\text{Double Precision})$$

FLOATING POINT NUMBERS

DECIMAL TO IEEE 754 FLOATING POINT:

EXAMPLE 1: Convert $(1460.125)_{10}$ to 32-bit single precision IEEE 754 binary floating point number. Also find in double precision.

SOLUTION: Step 1: Conversion from decimal to binary

$$(1460.125)_{10} = (10110110100.001)_2$$

Step 2: Normalization:

$$10110110100.001 = 1.0110110100001 * 2^{10}$$

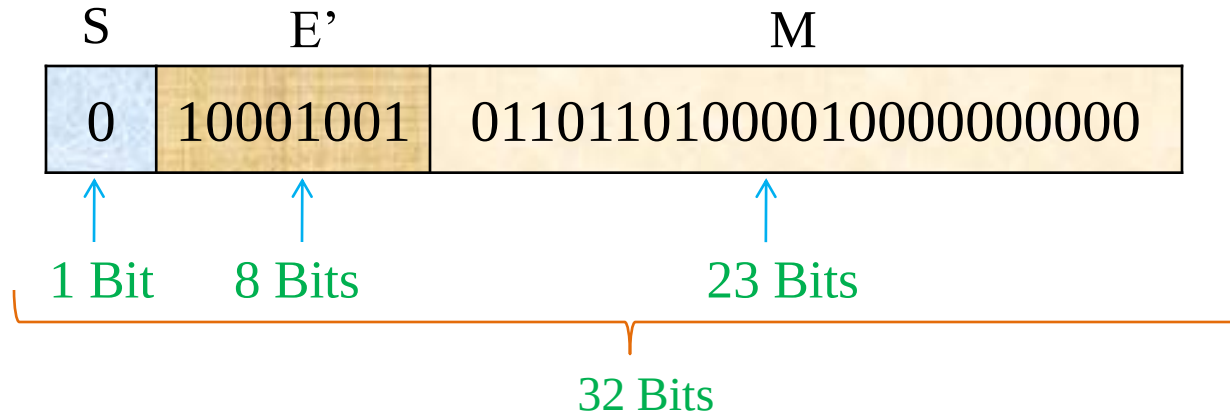
Step 3: $S = 0$,

$$E' = E + 127 = 10 + 127 = 137$$

$$(137)_{10} = (10001001)_2$$

FLOATING POINT NUMBERS

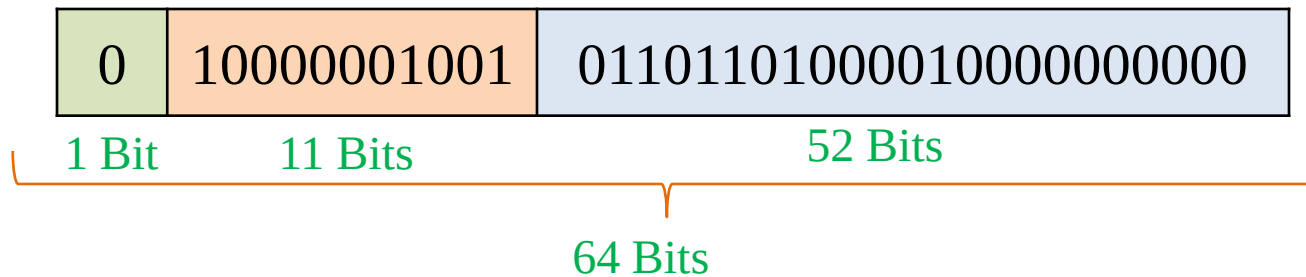
Step 4: Single Precision



Step 5: Double Precision

$$S = 0, \quad E' = E + 1023 = 10 + 1023 = 1033$$

$$(1033)_{10} = (10000001001)_2$$



FLOATING POINT NUMBERS

IEEE 754 FLOATING POINT TO DECIMAL CONVERSION:

SINGLE PRECISION:



$$X_{\text{FP32}} = (-1)^s * 2^{e-127} * 1.M$$

DOUBLE PRECISION:



$$X_{\text{FP32}} = (-1)^s * 2^{e-1023} * 1.M$$

FLOATING POINT NUMBERS

EXAMPLE 2: Convert into decimal number.

1	10000011	01111110000000000000000000000000
---	----------	----------------------------------

SOLUTION:

$$s = 1$$

$$(10000011)_2 = (131)_{10} = e$$

$$M = 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4} + 1 \times 2^{-5} + 1 \times 2^{-6} + 1 \times 2^{-7}$$

$$M = 0.4921875$$

Using formula $X_{\text{FP32}} = (-1)^s * 2^{e-127} * 1.M$

$$= (-1)^1 2^{131-127} * 1.4921875$$
$$= -23.875$$

FLOATING POINT NUMBERS

EXAMPLE 3: What is decimal equivalent of the floating point number

40400000H

SOLUTION:

40400000H =

0	100 0000	0	100 0000 0000 0000 0000 0000
s	e		M

$$S = 0$$

$$e = (10000000)_2 = (128)_{10}$$

$$M = 1 * 2^{-1} + 0 * 2^{-2} + \dots = 0.5$$

$$X_{\text{FP32}} = (-1)^s * 2^{e-127} * 1.M = (-1)^0 2^{128-127} * 1.5 = 3_{10}$$

FLOATING POINT NUMBERS

IEEE754 Format has following special cases

e	m	inference
255	0	Not a Number (NaN)
255	$\neq 0$	Infinite number
0	0	$X = (-1)^s * 2^{-126} * (0.m)$
0	$\neq 0$	0, $(-1)^s * 0$, again +0 and -0 are possible

THANK YOU

shasansaeed@yolasite.com

hasansaeed872726549.wordpress.com

saeed.moodlecloud.com

syedhasansaeed.gnomio.com